Politechnika Krakowska
Faculty of Computer Science and Mathematics
Problem Solving — Academic Year 2025/2026

Travel Planner Application

Project Report

Pablo Cervera Moreno
Diego Perez Cuno

1. Introduction

This report describes the design, implementation, and testing of the Travel Planner Application, developed as the final project for the Problem Solving course at Politechnika Krakowska during the Spring Semester of the 2025/2026 academic year by Pablo Cervera Moreno and Diego Perez Cuno.

The goal of the project was to build a client-side web application that allows users to search for tourist attractions in any city, filter and sort results by category, save places to a personal Trip List, and add custom locations manually. The application runs entirely in the browser with no back-end server and relies on free, open-source public APIs.

1.1 Objectives

- Search for places and attractions by city name using a public API
- Display results with name and category badge
- Allow users to add and remove places from a persistent Trip List
- Support multi-filtering by category and sorting by name
- Implement a simple API cache using LocalStorage to reduce redundant requests
- Provide a form for adding custom places with client-side input validation
- Include unit tests covering core application logic

1.2 Project Scope

The application is a pure front-end solution: a single HTML page, one CSS stylesheet, and one JavaScript file. No build tools, frameworks, or back-end infrastructure are required. The app is deployed on GitHub Pages and accessible at https://pablitoCM.github.io/travel-planner/

2. Technical Design & Architecture

2.1 Technology Stack

Technology	Role
HTML	Page structure and semantic markup
CSS	Responsive layout using Grid, Flexbox and custom properties
JavaScript	Application logic, API calls, DOM manipulation
Fetch API	Asynchronous HTTP requests to external APIs
LocalStorage	Persistent Trip List and API response cache
Nominatim (OpenStreetMap)	City name geocoding — free, no API key required
Overpass API (OSM)	POI data retrieval — free, no API key required

2.2 API Integration

A key design decision was to use only APIs that require no registration or API key, so the project runs immediately without any configuration. Two services from the OpenStreetMap ecosystem were selected:

- Nominatim: accepts a free-text city name and returns coordinates plus a bounding box for the city boundaries.
- Overpass API: accepts a structured query in Overpass QL and returns all OSM nodes matching given tag filters within that bounding box.

The search flow works as follows:

- The user types a city name and clicks Search.
- geocodeCity() calls Nominatim to resolve the city to coordinates and a bounding box.
- fetchPlaces() builds an Overpass QL query and sends it as a POST request.
- Results are normalised into a common internal shape and stored in allResults.
- applyFilters() filters and sorts the array before rendering.

2.3 LocalStorage Cache

Search results are cached in LocalStorage with a TTL of 10 minutes. The cache key is built from the city name and the active category filter, so different filter combinations are cached independently.

Property	Detail
Cache key format	tp_cache_<city>_<category>

Property	Detail
TTL	10 minutes (600,000 ms)
Storage format	JSON: { ts: timestamp, data: [...places] }
Expiry check	Date.now() - ts > CACHE_TTL_MS → treat as miss
Failure handling	Write failures silently ignored (quota exceeded, private mode)

3. Features & Implementation

3.1 Search

The user types a city name in the hero search bar and presses Search or Enter. While loading, skeleton placeholder cards are displayed. On completion, result cards appear showing the place name and a category badge.

3.2 Filtering and Sorting

A sticky filter bar appears after the first search. Each category option maps to specific OSM tag values through the FILTER_MATCHERS dictionary, which contains a predicate function per category. Changing the filter re-runs applyFilters() on the already-loaded allResults array without making a new API call, giving instant response. Results can also be sorted alphabetically in both directions.

3.3 Trip List

The Trip List panel shows saved places and is persisted in LocalStorage, surviving page reloads. Users can sort the list by addition order, name, or category. Adding a duplicate is rejected with a toast notification.

3.4 Detail Modal

Clicking any result card opens a modal with all available OSM data for that place: category, coordinates, opening hours, phone number, entrance fee, and links to Google Maps and OpenStreetMap. A Wikipedia link is shown when the OSM record includes a wikipedia tag. The place can be added to the Trip List directly from the modal. The modal closes on the X button, backdrop click, or Escape key.

3.5 Custom Place Form

The Add Custom Place panel lets users add locations that may not appear in the API results. The form requires a place name (minimum 2 characters) and a category, with an optional city field. Client-side validation displays inline error messages without losing the user's input.

3.6 Toast Notifications

Feedback messages are shown as a toast notification at the bottom of the viewport and auto-dismiss after 2.5 seconds.

4. Design Decisions

4.1 Overpass API Instead of OpenTripMap

OpenTripMap was evaluated as an initial candidate but discarded in favour of the Overpass API for two concrete reasons.

First, real-world usage of OpenTripMap requires obtaining and managing an API key. Although a free tier exists, this adds configuration friction that conflicts with the project goal of a zero-setup application that runs immediately without any credentials.

Second, the metadata returned by OpenTripMap is limited compared to what OSM nodes carry natively. The Overpass API provides opening hours, phone numbers, website URLs, and Wikipedia tags out of the box, enabling the detail modal without any secondary requests.

Staying within the OpenStreetMap ecosystem also meant that Nominatim (geocoding) and Overpass (POI retrieval) share the same underlying data model, simplifying the normalisation step. The trade-off is the need to learn Overpass QL, a domain-specific query language, which added initial development time but produced a more capable and fully open result.

4.2 Client-Side Filtering

When a category filter is applied after an initial search, the application does not make a new API call. It runs the FILTER_MATCHERS predicate functions over the already-loaded allResults array, giving instant feedback and avoiding unnecessary network requests. The initial load fetches up to 80 nodes across all categories, which is an acceptable trade-off.

4.3 State Management Without a Framework

The application maintains two module-level arrays: allResults (full result set from the last API call) and filteredRes (current view after filters and sorting). Any change to either array triggers renderResults() or renderTrip(), which rebuilds the relevant DOM section. This is simple, predictable, and sufficient for this scale.

4.4 HTML Escaping

All data from the API and from user input is passed through escHtml() before being inserted into innerHTML, preventing XSS attacks if a place name or description contains HTML special characters.

5. Testing

5.1 Unit Tests

The application includes a self-contained unit test suite accessible via runTests() in the browser console (F12 → Console → runTests()).

Function	What is tested	Assertions
sortResults()	A→Z and Z→A sort order on search results	3 assertions
sortTrip()	Name and category sort on the Trip List	2 assertions
cacheGet/cacheSet	Cache miss, cache hit, and TTL expiry	3 assertions
kindLabel()	OSM tag-to-readable-label conversion	4 assertions
Duplicate guard	Prevents adding the same place twice	2 assertions

Total: 14 assertions.

5.2 Manual Testing

Manual testing was conducted by searching multiple cities (Paris, Madrid, Warsaw, Tokyo) and verifying:

- Results are returned and cards render correctly
- Category filter shows only the matching places
- Sorting by name works in both directions
- Adding and removing from the Trip List persists across page reloads
- The custom form rejects empty name and missing category with inline errors
- The modal opens with correct details and map links work
- The cache is used on the second identical search
- The app is responsive on mobile viewports

6. Known Limitations & Future Work

6.1 Current Limitations

- OSM data quality varies by city — less-mapped cities may return fewer results.
- The Overpass API has rate limits; very rapid repeated searches may trigger a temporary block.
- Place descriptions are not available from Overpass. Wikipedia extracts could be fetched via the Wikimedia API.
- The Trip List is local to the device and browser — there is no account system.

6.2 Possible Future Enhancements

- Embedded interactive map (Leaflet.js + OpenStreetMap tiles) to show results geographically
- Wikipedia API integration to fetch short descriptions for detail modals
- Trip export to PDF or shareable URL
- Distance-based sorting using the Haversine formula

7. Conclusions

The most significant technical decision was the deliberate choice of Overpass API over OpenTripMap — prioritising zero-configuration setup and richer metadata over a more widely known alternative. This required learning Overpass QL from scratch but produced a more capable and fully open result.

The project provided practical experience with real problems in web development: API selection and trade-off analysis, state management without a framework, data normalisation from external sources, and defensive coding practices. These are skills that transfer directly to professional front-end development.

8. References

- MDN Web Docs — Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- MDN Web Docs — localStorage: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage
- Nominatim API: https://nominatim.org/release-docs/develop/api/Search/
- Overpass API: https://wiki.openstreetmap.org/wiki/Overpass_API
- Overpass QL: https://wiki.openstreetmap.org/wiki/Overpass_API/Overpass_QL
- OpenStreetMap tag reference: https://wiki.openstreetmap.org/wiki/Map_features
- GitHub repository: https://github.com/pablitoCM/travel-planner
- Live demo: https://pablitoCM.github.io/travel-planner/